

Backend Engineering Assignment

Food Delivery Order Processing Service

Role: Backend Developer (Go) @ Vaurd **Est. effort:** ~6–10 hours **Suggested window:** 5 days from receipt

1. Context

You'll build a small but realistic order-monitoring backend — the kind of system that powers a food-delivery operations dashboard.

Orders arrive continuously and change state continuously (status updates, item changes). Your job is to build a service that ingests this constant event stream, keeps each order's state current, and exposes an API that always returns the latest order information.

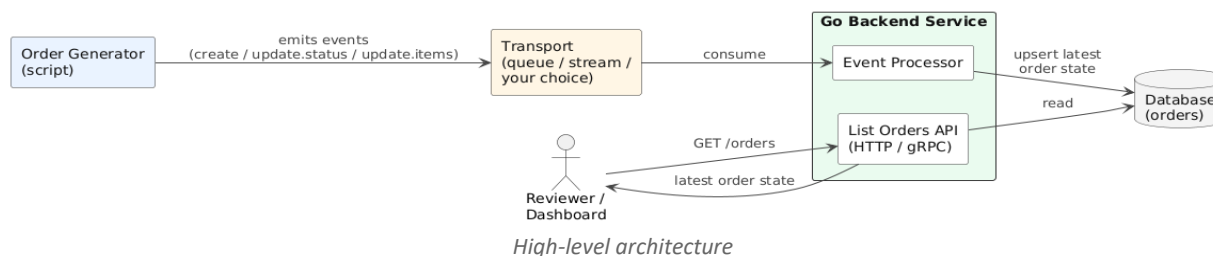
This mirrors the kind of event-driven, runtime-state problems we solve at Vaurd — so we care as much about *how you reason about the design* as about the working code.

2. Problem Statement

Build two things:

1. **An order event generator** — a script that continuously emits random food-order events (creations + updates).
2. **A Go backend service** — consumes those events, processes them, persists current order state to a database, and serves a List Orders API returning the latest state of every order.

Between them sits a transport of your choice (message queue, stream, or direct call — your design decision, justify it).



3. Event Specifications

The system receives **three event types**. The data payloads below are required; you design the surrounding event envelope (event id, type, timestamp, etc.).

3.1 order.create

Creates a new order. **Your service generates the orderId.**

Field	Type	Notes
customerId	string	
restaurantId	string	
items	array	list of { itemId: string, qty: number }

3.2 order.update.status

Updates the status of an existing order.

Field	Type	Notes
orderId	string	references an existing order
status	string (enum)	Received Preparing Complete Cancelled

3.3 order.update.items

Replaces / updates the items on an existing order.

Field	Type	Notes
orderId	string	references an existing order
items	array	list of { itemId: string, qty: number }

4. Requirements

4.1 Order Generator (script)

- Continuously generates a mix of all three event types with randomized data.
- Update events should reference orders that actually exist in the stream.
- Rate should be configurable (or at least sensible) so reviewers can watch the system work.
- Language is **your choice** — **any language** (only the backend service must be Go).

4.2 Backend Service (must be Go)

- Consumes and processes the event stream.
- Persists and maintains current order state in a database.
- Exposes a List Orders API returning the latest state of all orders.
- Handles a continuous stream — not a one-shot batch.

4.3 List Orders API

Returns the current state of each order. At minimum, per order: orderId, customerId, restaurantId, items, status, and a last-updated timestamp.

- **HTTP or gRPC** — your choice.
- **Bonus:** pagination, filtering by status, sorting by last-updated.

5. Design Considerations

We deliberately left edge cases open. **Make a decision, implement it, and justify it in DESIGN.md.** These are exactly what we'll dig into during the interview:

- **Out-of-order events** — an update may arrive for an order you haven't "created" yet. What do you do?
- **Concurrency** — many events for the same order can arrive close together. How do you keep state correct?
- **Duplicate / replayed events** — is your processing idempotent?
- **Status transitions** — is any transition valid (e.g. Complete → Preparing)? Do you enforce rules?
- **Throughput** — the stream never stops. How does your design hold up as volume grows?
- **"Latest" guarantees** — how do you ensure the List API reflects the most recent state?

There's no single right answer — we're evaluating your reasoning and trade-offs.

6. Technical Constraints

Area	Constraint
Backend language	Go (mandatory) — applies to the backend service; the generator can be any language
Database	Any (Postgres, Mongo, Redis, SQLite... your choice)
Transport / queue	Any (Kafka, NATS, RabbitMQ, channels, HTTP... your choice)
API style	HTTP or gRPC
AI tools	Allowed for development — but you'll be asked to explain your implementation in depth.

7. Deliverables

A **GitHub repository** containing:

File	Contents
(source code)	The Go backend service + the order generator
README.md	How to run the backend service · how to run the generator · prerequisites
API.md	Full API specification (endpoints, request/response schemas, examples)
DESIGN.md	Full design writeup — architecture, data model, event flow, and your answers to the design considerations in §5
docker-compose.yml	If your service needs dependencies (DB, broker, etc.), it should come up cleanly with this

Aim for something a reviewer can clone and run with minimal friction.

8. Evaluation Criteria

What we look for, roughly in priority order:

3. **Correctness** — does it process events and reflect accurate, latest state?
4. **Design & reasoning** — quality of trade-offs, clarity of DESIGN.md, handling of \$5 edge cases.
5. **Go fundamentals** — idiomatic code, concurrency handling, error handling, structure.
6. **Runnability** — clean setup, working docker-compose, clear docs.
7. **API quality** — sensible, well-documented, correct.

We value clear thinking and clean, working code over feature count. A simpler solution you fully understand beats a complex one you can't explain.

Good luck — we're looking forward to seeing how you think.